

SafetyFirst

Judge Q&A study guide — the web and backend side

Team Mojito — Bhavesh Waghmare, Jasbir Singh Monga, Priyal Vairagade. FAR AWAY 2026.

What this is	Every answer here was read out of the actual code, not written from memory.
Who it is for	The person answering questions about the website, API, and database.
How to use it	Read sections 1-4 until you can say them without looking. The rest is lookup.

A judge does not test whether you typed the code. They test whether you understand the system you are presenting and whether you made real decisions. Every answer below gives you the fact and the reason, because 'why' is the follow-up you will actually get.

START HERE

1. The answer you give first

If you only remember one thing, remember this shape. Everything else in the guide hangs off it.

Q. What is SafetyFirst, in one breath?

It is a PPE-compliance access-control checkpoint. A camera watches the entry gate, a computer-vision model checks whether the person is wearing the safety gear the site requires, and the gate opens or refuses on that basis. Every refusal is logged with the evidence behind it, so the site has a record it can be held to.

The important word is access control, not detection. Plenty of projects detect a helmet. Ours makes a decision, acts on it, and keeps a defensible record of why.

Q. What does the web side actually do in that sentence?

Three things. It is the decision-maker: the browser sends camera frames to the backend, the backend runs the model and applies site policy, and returns a verdict. It is the record: every gate decision, hazard alert, and policy change is stored in a database. And it is the control surface: an administrator sets what the gate demands, reviews refusals, and manages workers from a web console.

Q. How big is it, concretely?

39 API endpoints across 4 blueprints, 8 database tables, and 16 frontend pages. Flask and SQLAlchemy on the backend, vanilla JavaScript on the frontend with no build step.

Numbers count what is in the repo now. If a judge asks and you are unsure, say 'roughly 40 endpoints and 8 tables' rather than guessing precisely.

ARCHITECTURE

2. How the whole thing fits together

Q. Walk me through the architecture.

Four parts. (1) A browser page opens the camera and captures frames. (2) A Flask backend receives each frame, runs the YOLOv8 model on it, applies site policy, and returns a verdict plus the detection boxes. (3) A database stores the decisions, the alerts, the policy, and the audit trail. (4) An ESP32 sensor board reports environmental hazards (gas, temperature, humidity) into the same backend over WiFi.

The gate itself is driven by the verdict. A critical hazard from the ESP32 overrides the PPE decision entirely.

Q. Why is the backend split into four blueprints?

A blueprint is Flask's way of grouping related routes. Ours split by who is calling and what they are allowed to do, which makes the permission rules obvious instead of scattered.

auth_bp	/api/auth	sign up, log in, guest, Google
detection_bp	/api	frames, verdicts, live state, history
gate_bp	/api/gate	what a device reports: badges, sensors, alerts
admin_bp	/api/admin	admin-only: settings, users, analytics, audit

The rule that falls out of it: anything under /api/admin requires an admin account. Anything under /api/gate is a device stating a fact, so it only needs a valid session.

Q. Why vanilla JavaScript instead of React?

Because there is no build step to break at a demo. The frontend is 16 HTML pages that load Tailwind from a CDN and plain JavaScript modules. You can open any page as a file and it works. On a hackathon timeline, a broken npm build twenty minutes before judging is a bigger risk than the convenience React would have bought.

The honest tradeoff: shared logic is duplicated more than it would be in a component framework. We factored the common parts into js/shell.js and js/auth.js to limit that.

THE MOST LIKELY QUESTION

3. The core flow: camera to gate decision

This is the question you are most likely to get, and the one where a vague answer hurts most. Learn this path end to end.

Q. What happens between the camera seeing someone and the gate opening?

Step by step:

1. Browser grabs a frame from the <video> element
2. Encodes it as base64 JPEG, POSTs to /api/socket
3. Backend decodes it (cv2.imdecode) into an image array
4. YOLOv8 runs on that frame -> list of detections
5. evaluate_access() turns detections into a verdict
6. If the verdict CHANGED, it is saved to the database
7. JSON returns to the browser: verdict + boxes
8. Browser draws the overlay and speaks the result

The model never runs in the browser. The browser only captures and draws; every decision is made server-side, which is what makes the record trustworthy.

Q. How exactly is the verdict decided?

In evaluate_access(), in backend/detection.py. It checks three things in a deliberate order:

1. No person in frame -> 'no_person' (not a refusal)
2. Active critical hazard -> 'alert_hold' (overrides everything)
3. Compare detected PPE
against required_ppe -> 'granted' or 'denied'

The order matters and judges do probe it. An empty checkpoint must not log a stream of false violations, so 'no person' is checked first. And the hazard check sits above the PPE comparison deliberately: someone in full gear is still not safe to admit into a gas leak, so a clean pass must not be able to produce a GRANTED underneath an active alarm.

Q. The camera sends many frames a second. Do you save a record for every frame?

No, and this is a design decision worth stating out loud. A person standing at the gate produces the same verdict dozens of times. Saving each one would bury the real entries under hundreds of duplicates and make the log useless.

So we only write a record when the verdict actually changes. A gate decision is an event, not a frame. In the code that is the is_new_decision check.

Q. What decides which PPE is required?

Site policy, stored in the database and editable by an admin from the Settings page. The default is Hardhat and Safety Vest. It is read per frame, not once at startup, so an administrator can change what the gate demands while it is running and the very next frame honours it.

It is cached in memory with a lock, because a database round-trip on every frame would cost more than the inference does.

Q. What is the confidence threshold and why is it a setting?

Below that confidence a detection is treated as noise. The default is 0.25. It is a configurable setting rather than a constant because it is genuinely site policy: raising it suppresses false violations but risks missing marginal real ones. That is a safety tradeoff a site supervisor should own, not something hardcoded by us.

Q. You store the policy with each decision. Why?

Because a record that says 'denied' is worthless six months later if you cannot show what the rules were at that moment. Each DetectionRecord saves a policy_json snapshot of the required PPE and confidence threshold in force when the decision was made. If an admin later weakens the policy, old refusals still explain themselves.

4. The database

If you get asked to explain the schema, this table is the answer. The pattern to notice: things that are read constantly are kept separate from things that must never be lost.

Table	What it holds	Why it exists separately
users	Accounts and worker profiles	Badge, role, photo shown at the gate
site_settings	Required PPE, thresholds, location	Policy an admin changes without a redeploy
attendance_records	One row per badge scan	Who presented themselves, and were they let in
detection_records	One row per gate decision	What the camera saw, plus policy at the time
sensor_readings	Latest value per sensor kind	Overwritten each report; the live readout
sensor_reading_log	Every value ever reported	The trend history behind the live number
sensor_alerts	Hazards that crossed a threshold	The durable record of what fired and when
audit_events	Every policy change	Append-only; who changed what, and when

Q. Why are attendance and detection separate tables?

They answer different questions. Attendance logs that a named person presented themselves and whether they were admitted. Detection logs what the camera actually saw. Attendance is the record a supervisor asks for; detections are the evidence behind it. Merging them would mean you cannot produce one without dragging in the other.

Q. Why keep both sensor_readings and sensor_reading_log?

sensor_readings holds one row per kind, overwritten on every report; it answers 'what is the gas level right now'. sensor_reading_log appends every reading; it answers 'what has it been doing'. The live panel needs the first, the trend chart needs the second. One table cannot do both cheaply.

The log is capped at 2000 rows per kind with the oldest pruned automatically, because the sensor reports every few seconds forever and an uncapped table would grow without bound.

Q. Is the audit log editable?

No, deliberately. There is no endpoint to edit or delete an audit entry. A log an administrator can quietly rewrite proves nothing. That is why it is append-only by design rather than by convention.

Q. SQLite or a real database?

SQLite for local development, PostgreSQL in production via a DATABASE_URL environment variable. The code handles both. There is a documented caveat in config.py: SQLite on an ephemeral container filesystem gets wiped on redeploy, so production must point at managed Postgres or the compliance record does not survive.

Good answer to 'is this production-ready'. You knew the failure mode and wrote it down rather than discovering it live.

5. Authentication and permissions

Q. How does login work?

JWT, using flask-jwt-extended. You sign in, the server returns a signed token, and the browser stores it and sends it on every request as an Authorization: Bearer header. The server verifies the signature rather than looking up a session, so it holds no session state. Tokens last 7 days.

Four ways in: email and password signup, login, Google sign-in (verified against Google's own token endpoint with our client ID), and a guest account.

Q. What is the guest account for?

A visitor or a device that needs a valid session but not an identity. It is created with no credentials and its token expires in 24 hours instead of 7 days. The ESP32 currently signs in this way, which is how it can post sensor readings without us embedding a real password in the firmware.

Be ready for the follow-up in section 9 — this is also the weakest point in the system and you should own it before a judge finds it.

Q. How are passwords stored?

Hashed with Werkzeug's generate_password_hash, never in plain text. The login path compares hashes. There is also a deliberate detail: a wrong email and a wrong password return the same error message, so the endpoint does not tell an attacker which accounts exist.

Q. What separates an admin from a normal user?

An is_admin flag on the user row, enforced by an admin_required decorator on every route under /api/admin. It checks the flag on the account behind the token, not something the client sends, so a user cannot promote themselves by editing a request.

ESP32 INTEGRATION

6. The sensor side, from the web's point of view

You know the hardware. This section is only about how it meets the backend, which is the part a judge will aim at you rather than at the hardware answer.

Q. How does the ESP32 talk to the backend?

Plain HTTP over WiFi, exactly like any other client. It signs in once to get a JWT, then POSTs JSON to /api/gate/sensors with a reading. Same authentication path as the browser; nothing special was built for it.

```
POST /api/gate/sensors
Authorization: Bearer <token>
{ "kind": "gas", "value": 450, "source": "esp32" }
```

If the token expires or the backend restarts, the board gets a 401 and signs in again automatically, once, before retrying. That means the sensor recovers on its own instead of needing a reboot.

Q. Does the sensor decide what counts as dangerous?

No, and that is the important design point. The board reports a number. The backend compares it against a threshold an admin configured for that sensor kind and decides whether it is nothing, a warning, or critical.

The reason: thresholds are site policy and they change. If the board decided severity, changing the gas limit would mean reflashing firmware. This way it is a form field on the Alerts page.

Q. What if a sensor reports a kind nobody configured a threshold for?

It is stored and shown in Live Readings, and raises nothing. That is intentional, not an oversight: a new sensor should be able to start reporting before anyone has decided what a dangerous value is, without either crashing or firing false alarms.

Q. Is the ESP32 real, or simulated?

The board is real and on the network; the sensor values are currently simulated in firmware because the gas sensor and DHT11 are not wired yet. The simulator does a random walk around a baseline with occasional spikes, so values drift the way a real sensor does instead of jumping randomly.

What that proves is the part that matters: the network path, the authentication, the threshold logic, and the alert pipeline are all real and tested end to end. When the physical sensor arrives, only the line that produces the number changes. No backend change at all.

Say this plainly if asked. Claiming a sensor you do not have is the fastest way to lose a judge's trust; explaining exactly which part is stubbed and why the rest is real usually gains it.

7. Hazard alerts

Q. What happens when gas crosses the threshold?

The backend classifies the reading as warning or critical. A critical alert immediately holds the gate: `evaluate_access()` returns `alert_hold` for every frame until it is cleared, regardless of what anyone is wearing. It also appears as a banner on every open admin and operator page.

A warning is a heads-up only and does not hold the gate.

Q. The sensor reports every few seconds. Does that create an alert every time?

No. There is at most one active alert per kind. If gas is still above the limit on the next reading, the existing alert is returned instead of a new one being created. Otherwise one real leak would produce hundreds of duplicate rows.

Q. I acknowledged an alert and it came straight back. Is that a bug?

No, that is correct behaviour and it is worth being able to explain calmly. Acknowledging means a human saw it. It does not lower the gas level. If the value is still above the threshold on the next reading, a new alert fires.

For a real gas sensor that is exactly what you want: clearing the alarm must not silence a leak that is still happening.

Q. Who can clear a hazard alert?

Any signed-in user, not just an admin. That was a deliberate call: the person standing at the gate when the air clears is usually not the administrator, and making them wait for one is worse than the risk of a premature clear. Every acknowledgement is recorded with the name of who did it, so the decision is attributable rather than anonymous.

This is a good 'decision we would defend' answer. It shows you weighed two bad options instead of not noticing there was a choice.

THE ANSWER THAT IMPRESSES

8. Evidence and privacy

Q. Do you store images of people?

Only of refusals, never of clean passes. When someone is turned away, the frame behind that decision is saved so the refusal can be reviewed and challenged. If the gate opened, there is nothing to justify, so no image is kept.

Q. Isn't storing photographs of workers a privacy problem?

It is, which is why the images expire. Refusal frames are deleted after 30 days by default, configurable via `EVIDENCE_RETENTION_DAYS`. The decision record itself is kept permanently; only the photograph ages out.

The reasoning: the system needs to be able to justify a refusal, but it should not quietly build a permanent photographic record of every worker's bad day. Those are different needs and they deserve different retention.

Q. Is this face recognition?

No. Identity comes from a badge or RFID scan, not from the face. The camera classifies safety equipment, not people. That was a deliberate choice: face recognition at a workplace entrance is a much heavier thing to deploy, legally and ethically, and it is not needed to answer the question we are actually asking, which is whether this person is wearing a helmet.

Q. Where are the images stored?

On disk, not in the database. They are roughly 50KB JPEGs written on every refusal, and storing them as database blobs would bloat the backups that exist to protect the decision record itself.

PREPARE FOR THESE

9. The hard questions

These are the ones designed to find out whether you understand the system or just built it. An honest, specific answer beats a confident vague one every time.

Q. What is the weakest part of this system?

Device authentication. Right now the ESP32 signs in as a guest, and a guest account can be created with no credentials. That means anyone who can reach the API could raise a critical alert and hold the gate. We found it in our own security review and it is a known, written-down gap rather than something we missed.

The fix is a device account with a shared secret; the firmware already has the fields for it (DEVICE_EMAIL and DEVICE_PASSWORD). We left it as guest because the sensor board is still on the bench and rotating credentials mid-development costs more than it protects here.

Volunteering this makes you look rigorous. Being caught not knowing it makes you look careless. Say it before they find it.

Q. How do you know the code actually works?

We ran a security review over the backend and fixed what it found, verifying each issue by reproducing it against the running server rather than reading code and guessing. Concrete example: a sensor could report a value of infinity, which passed the number check but serialised to JSON the browser refuses to parse, permanently breaking the live readings panel for everyone until the row was deleted by hand. We reproduced it, fixed it, and re-tested that the exploit no longer works.

We also fixed a timezone bug where the server sent timestamps without a UTC marker, so the browser read them as local time and showed a three-second-old reading as six hours old.

Q. What happens if the model is wrong?

Two mitigations and one honest limit. The confidence threshold filters low-certainty detections, and refusals keep the frame so a wrong refusal can be reviewed and challenged by a human. The limit: a false negative, letting someone through who should not be, leaves no evidence, because we do not photograph clean passes. That is a real tradeoff we made in favour of privacy.

Q. Why should the gate trust a browser? Could someone fake frames?

They could, and that is a genuine limitation of the current demo setup, where the camera page runs in a browser. The mitigation in the intended deployment is that the checkpoint runs on a dedicated Raspberry Pi at the gate, not on a machine a worker has access to. The verdict is always computed server-side, so a client cannot simply declare itself compliant; it can only supply pixels.

Q. What would you do with another month?

In order: device authentication with per-device credentials, so the sensor path is not guest-authenticated. Then getting the model onto the Raspberry Pi AI HAT, because inference currently runs on a CPU. Then a real hardware gate relay, since right now the decision is shown and spoken rather than physically opening a barrier.

Q. How is this different from just running YOLO on a webcam?

A model tells you what is in a frame. This decides what to do about it, enforces a policy someone chose, refuses entry, keeps evidence for the refusals, records who changed the rules, and escalates when the air itself is dangerous. The detection is maybe fifteen percent of what is here. The rest is the part that would let a site actually rely on it.

10. If you blank out

Numbers and names worth having on the tip of your tongue.

Thing	Answer
Backend	Python, Flask, SQLAlchemy ORM
Frontend	Vanilla JavaScript, Tailwind via CDN, no build step
Auth	JWT (flask-jwt-extended), 7-day tokens, 24h for guests
Model	YOLOv8 (Ultralytics), loaded from best.pt
Default policy	Hardhat + Safety Vest, confidence 0.25
Database	SQLite in dev, PostgreSQL in production
Scale	39 endpoints, 8 tables, 16 pages, 4 blueprints
Frame endpoint	POST /api/socket (base64 JPEG in, verdict + boxes out)
Verdicts	granted, denied, no_person, alert_hold
Sensor endpoint	POST /api/gate/sensors (raw value, backend classifies)
Evidence retention	30 days, refusals only, configurable
Audit log	Append-only, no edit or delete endpoint exists

Three sentences that answer most questions

1. "The browser only captures and draws; every decision is made on the server, which is what makes the record trustworthy."
2. "That is site policy, not something we hardcoded, because it is the kind of thing a supervisor needs to change without a redeploy."
3. "We only record it when the decision changes, because a gate decision is an event, not a frame."

If you genuinely do not know

Say so, then say where the answer lives. "I do not remember the exact number, but that is handled in the alerts module and it works like this..." is a strong answer. Inventing a figure is the one thing that will actually cost you, because judges follow up.